

Стать IT-шником может каждый



Математические операции над NumPy массивами



Преподаватель



Ваше фото

ИМЯ ФАМИЛИЯ

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)



ВАЖНО!

- Камера должна быть включена на протяжении всего занятия.
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях.
- Вести себя уважительно и этично по отношению к остальным участникам занятия.
- Во время занятия будут интерактивные задания, будьте готовы включить микрофон или демонстрацию экрана по просьбе преподавателя.

ПЛАН ЗАНЯТИЯ

Введение

Основной блок

Задание для закрепления

Вопросы по основному блоку

Заключение



Стать IT-шником может каждый



ПОВТОРЕНИЕ ИЗУЧЕННОГО





Повторение

- Что такое NumPy
- Создание массивов
- Работа с размерностями
- Типы элементов в массивах



Стать IT-шником может каждый



ВОПРОСЫ





Введение

Базовые операции с массивами

Другие арифметические операции над массивами и `inf/nan`

Сравнения, различные встроенные функции



Стать IT-шником может каждый



ОСНОВНОЙ БЛОК



Стать IT-шником может каждый



Базовые операции с массивами



Базовые операции со списками Python

```
two_d_list = [[1, 2, 3], [4, 5, 6]]  
# Сложение массивов  
print(two_d_list + two_d_list)  
# [[1, 2, 3], [4, 5, 6], [1, 2, 3], [4, 5, 6]]  
# Умножение массива на число  
print(two_d_list * 2)  
# [[1, 2, 3], [4, 5, 6], [1, 2, 3], [4, 5, 6]]
```

Базовые операции с массивами

Для `numpy.array` арифметические операции производятся поэлементно!

Базовые операции с массивами

```
two_d_array = np.array([[1, 2, 3], [4, 5, 6]])  
# Сложение массивов  
print(two_d_array + two_d_array)  
# [[ 2  4  6]  
#   [ 8 10 12]]  
# Умножение массивов  
print(two_d_array * two_d_array)  
# [[ 1  4  9]  
#   [16 25 36]]  
# Умножение массива на число – каждое число в массиве умножится на это число  
print(two_d_array * 5)  
# [[ 5 10 15]  
#   [20 25 30]]
```

Поэлементное умножение/деление

```
square = np.array([[1, 2], [4, 5]])  
two_d_array + square  
# ValueError: operands could not be broadcast together with shapes (2,3) (2,2)
```


Расширение размерности

```
one_d_array = np.array([10, 20, 30])
two_d_array = np.array([[1, 2, 3], [4, 5, 6]])
print(one_d_array + two_d_array)
# [[11 22 33]
#   [14 25 36]]
```

Добавление вектор-столбца

```
col = np.array([[10], [20]])  
two_d_array = np.array([[1, 2, 3], [4, 5, 6]])  
print(col + two_d_array)  
# [[11 12 13]  
#  [24 25 26]]
```

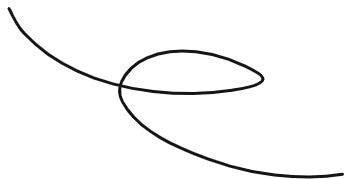
Объединение двух массивов

```
col = np.array([1, 2])  
two_d_array = np.array([3, 4, 5])  
print(np.concatenate( [col, two_d_array] ))  
# [1 2 3 4 5]
```



np.concatenate —

это функция из библиотеки numpy, принимающая список объединяемых массивов, а не метод массива.



Функция np.concatenate

```
two_d_array.concatenate(col)  
# AttributeError: 'numpy.ndarray' object has no attribute 'concatenate'
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Что будет выведено в следующих случаях::

1.

```
col = np.array([[10], [20], [30]])  
two_d_array = np.array([[1, 2, 3], [4, 5, 6]])  
print(col + two_d_array)
```
2.

```
mult = np.array([[10, 20, 30]])  
two_d_array = np.array([[1, 2, 3], [4, 5, 6]])  
print(mult * two_d_array)
```

Другие арифметические операции над массивами и `inf/nan`



Возведение массива в степень другого

```
a = np.array([0, 2, 1])  
b = np.array([3, 4, 5])  
print(a - b) # [-3 -2 -4]  
print(a / b) # [0. 0.5 0.2]  
print(a ** 2) # [0 4 1]  
print(a ** b) # [0 16 1]
```

Деление на 0

```
print(b / a) # [inf 2. 5.]  
# RuntimeWarning: divide by zero encountered in divide print(b / a)
```

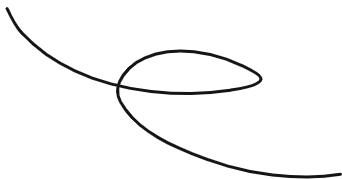
Деление на 0

```
print(type((b / 0)[0]))  
# <class 'numpy.float64'>
```



assert —

это инструкция, которая проверяет истинность стоящего после неё выражения и в случае его ложности выбрасывает исключение `AssertionError`.



np.inf

```
assert np.inf == (b / 0)[0]
```

Деление отрицательного числа на 0

```
print(np.array([-1])/0)  
# [-inf]  
print(np.array([1])/-0.)  
# [-inf]
```

Константа np.nan

```
print(np.array([0])/0)  
# [nan]  
print(np.array([np.inf])/np.inf)  
# [nan]
```

Операции над бесконечностью и нулями

```
print(np.array([np.inf])/0)
# [inf]
print(np.inf / 0.)
# ZeroDivisionError: float division by zero
print(0 / np.inf)
# 0.0
print(np.nan + 1, np.inf + 1, np.inf * 0, 1. / np.inf)
# nan inf nan 0.0
```

Стать IT-шником может каждый



ВОПРОСЫ



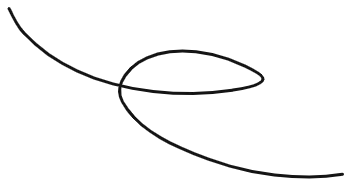
Сравнения, различные встроенные функции





Универсальные функции (ufunc) —

это элементарные функции, которые содержит библиотека numpy, они применяются к массивам поэлементно.



Математические функции

```
a = np.array([0, 2, 1])
print(np.sin, type(np.sin))
# <ufunc 'sin'> <class 'numpy.ufunc'>
print(np.sin(a))
# [0.      0.90929743 0.84147098]
```

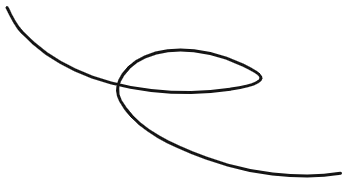
Булевы массивы

```
a = np.array([0, 2, 1])  
b = np.array([3, 2, 5])  
print(a > b)  
# [False False False]  
print(a == b)  
# [False True False]  
c = a >= 2  
print(c)  
# [False True False]
```



np.any —

это аналог `any` и `all` из классического Python, возвращает True, если хотя бы один элемент в массиве True, `np.all` – если все элементы в массиве True.



np.all

```
print(np.any(c), np.all(c))  
# True False
```

Модификация на месте

```
b *= 2  
print(b)  
# [ 6 4 10]
```

Набор математических операций

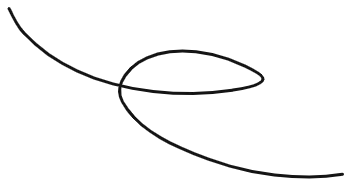
```
d = np.array([1, 5, 6])  
print(d.sum(), d.prod(), d.max(), d.min(), d.mean(), d.std())  
# 12 30 6 1 4.0 2.160246899469287
```

Полезно знать



Среднее значение —

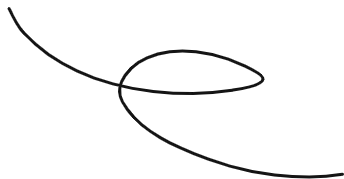
это среднее арифметическое всех чисел.





Среднее арифметическое —

для n чисел это их сумма, поделенная на их количество.
Является простейшей характеристикой ряда.



Стать IT-шником может каждый



Среднее арифметическое

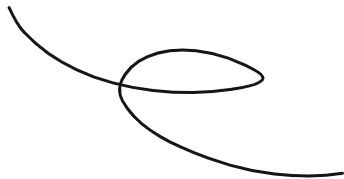


Для чисел 1, 5 и 6 среднее арифметическое равно $\frac{1+5+6}{3} = \frac{12}{3} = 4$



Среднеквадратичное отклонение —

это простейшая характеристика похожести элементов в ряде чисел. Оно равно корню из среднего квадрата отклонения чисел от среднего значения. Показывает, насколько числа имеют большой разброс относительно среднего значения.



Среднеквадратичное отклонение



Формула:

$$s = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}$$

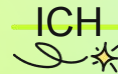
Посчитаем эту величину для чисел 1, 5 и 6:

$$\sum_{i=1}^3 (x_i - 4)^2 = (1 - 4)^2 + (5 - 4)^2 + (6 - 4)^2 = 9 + 1 + 4 = 14$$

Досчитаем в Python:

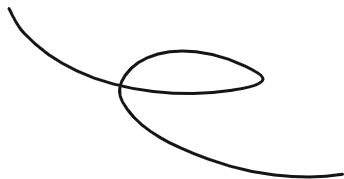
$$(14/3)**0.5 == 2.1602$$

Полезно знать



Дисперсия —

это квадрат среднеквадратичного отклонения.



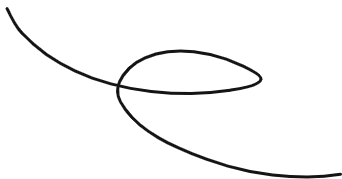
Полезные функции

```
print(np.sqrt(b))
print(np.exp(b))
print(np.log(b))
print(np.sin(b))
print(np.e, np.pi)
# [2.44948974 2.      3.16227766]
# [ 403.42879349  54.59815003 22026.46579481]
# [1.79175947 1.38629436 2.30258509]
# [-0.2794155 -0.7568025 -0.54402111]
# 2.718281828459045 3.141592653589793
```




Массив частичных сумм —

это массив, на первом месте которого стоит первый элемент, на втором – сумма первого и второго, на третьем – сумма первого, второго и третьего и т.д.



Массив частичных сумм

```
print(b.cumsum())  
# [ 6 10 20]
```


Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Что будет выведено в следующих случаях и почему:

```
cube = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(cube * cube)
print(cube + np.array([[10, 20], [30, 40]]))
print(cube + np.array([[10, 20]]))
print(cube + np.array([10, 20]))
print(cube + np.array([[10], [20]]))
```

Стать IT-шником может каждый



ПРАКТИЧЕСКАЯ РАБОТА



Практическое задание 1

1. Создайте массив из 20 чисел от -2π до 2π .
2. Посчитайте поэлементные квадраты косинусов для данного массива.
3. С помощью `pr.all` проверьте, что в ответе только числа не больше единицы.

Практическое задание 2

Создайте массив 4x4 со значениями от 0 до 15. Выведите:

1. Умножьте исходный массив на его третью строку
2. Вычтите из исходного массива второй столбец

Практическое задание 3

Что выведет код ниже:

```
print(np.array([0.0, 0.0, 1.0, -1.0]) / np.array([1.0, 0.0, 0.0, 0.0]))
```


Стать IT-шником может каждый



ОСТАВШИЕСЯ ВОПРОСЫ





Домашнее задание

1. Напишите программу, которая:

- Запрашивает у пользователя ввод двух одномерных массивов одинаковой длины. Массивы вводятся в виде чисел, разделенных пробелами.
- Преобразует введенные строки в массивы NumPy.
- Складывает два массива поэлементно.
- Выводит результат сложения на экран.





Домашнее задание

2. Напишите программу, которая:

- Запрашивает у пользователя размер массивов N .
- Генерирует два одномерных массива длины N с случайными целыми числами от 1 до 10.
- Выводит оба сгенерированных массива на экран.
- Выполняет поэлементное умножение массивов.
- Выводит результат умножения на экран.





Домашнее задание

3. Напишите программу, которая:

- Запрашивает у пользователя количество элементов массива N.
- Генерирует одномерный массив длины N, заполненный случайными числами от 0 до 100.
- Выводит сгенерированный массив на экран.
- Запрашивает у пользователя число X.
- Умножает каждый элемент массива на число X.
- Выводит измененный массив на экран.



Полезные ссылки

- [Основы NumPy: работа с массивами](#)
- [Индексация и срезы в NumPy](#)

Стать IT-шником может каждый



ЗАКЛЮЧЕНИЕ

